

Infron vs OpenRouter Routing, Cache, Cost, and Streaming Performance A/B Test Report

Abstract

This report evaluates `minimax/minimax-m2.7` under the standard Infron vs OpenRouter Prompt Caching A/B benchmark. Token cache hit rate: Infron leads in 2/4 routing modes; Observed cost: Infron leads in 4/4 routing modes; Throughput: OpenRouter leads in 3/4 routing modes; E2E latency: OpenRouter leads in 3/4 routing modes; TTFT: OpenRouter leads in 3/4 routing modes. The practical interpretation is that platform choice should be tied to the target objective instead of a single global metric.

The A/B filter retained 800 paired samples and 3200 request-level observations; 0 records were excluded. All core metrics come from response telemetry.

Conclusion Matrix

Routing mode	Cache hit rate	Observed cost	Throughput	E2E latency	TTFT
Throughput First	Infron	Infron	OpenRouter	OpenRouter	OpenRouter
Price First	Infron	Infron	Infron	Infron	Infron
Latency First	OpenRouter	Infron	OpenRouter	OpenRouter	OpenRouter
TTFT First	OpenRouter	Infron	OpenRouter	OpenRouter	OpenRouter

Overall Metrics

Routing mode	Platform	Pairs	Token cache hit rate	Actual cost	Avg throughput	Avg E2E latency	Avg TTFT
Throughput First	Infron	200	97.41%	\$0.36936000	2.67 tok/s	5980.69 ms	5786.47 ms
Throughput First	OpenRouter	200	93.54%	\$3.59489664	3.39 tok/s	4723.73 ms	4423.83 ms
Price First	Infron	200	98.58%	\$0.35053300	2.60 tok/s	6150.60 ms	5959.01 ms
Price First	OpenRouter	200	66.56%	\$0.72619654	1.79 tok/s	8930.26 ms	8499.25 ms
Latency First	Infron	200	98.43%	\$0.39538500	5.30 tok/s	3016.90 ms	2837.35 ms
Latency First	OpenRouter	200	98.90%	\$3.53757760	5.49 tok/s	2916.05 ms	2564.25 ms

Routing mode	Platform	Pairs	Token cache hit rate	Actual cost	Avg throughput	Avg E2E latency	Avg TTFT
TTFT First	Infron	200	98.43%	\$0.37421000	4.88 tok/s	3281.74 ms	2994.02 ms
TTFT First	OpenRouter	200	98.88%	\$3.63688800	5.56 tok/s	2875.68 ms	2440.43 ms

Prompt Length Stratification

Tier	Platform	Pairs	Token cache hit rate	Actual cost	Avg E2E latency	Avg TTFT
short (1500 target tokens)	Infron	268	77.71%	\$0.13344700	3632.64 ms	3421.09 ms
short (1500 target tokens)	OpenRouter	268	81.22%	\$0.46397534	3568.51 ms	3223.26 ms
medium (8000 target tokens)	Infron	268	98.91%	\$0.29008400	4367.24 ms	4173.93 ms
medium (8000 target tokens)	OpenRouter	268	87.49%	\$2.26869193	4592.34 ms	4219.52 ms
long (32000 target tokens)	Infron	264	99.05%	\$1.06595700	5840.97 ms	5605.71 ms
long (32000 target tokens)	OpenRouter	264	90.38%	\$8.76289151	6447.10 ms	6026.10 ms

Method And Reproducibility

The run used the standard 4 groups x 50 rounds design, streaming Chat Completions, platform–default reasoning/thinking behavior, prompt–length tiers `short:1500,medium:8000,long:32000` , and `/v1/chat/completions` only. A/B inclusion allowed up to 50 prompt–token difference within each paired first/second request.

Artifact	Path
Summary JSON	<code>export/minimax_m27_all_experiments/routing_sort_cache_cost_ab_4x50_stream_ttft_reasoning_default_length_tiers_chat_completions_summary.json</code>
Paired dataset CSV	<code>export/minimax_m27_all_experiments/routing_sort_cache_cost_ab_4x50_stream_ttft_reasoning_default_length_tiers_chat_completions_benchmark_pairs.csv</code>
Request–level JSONL	<code>export/minimax_m27_all_experiments/routing_sort_cache_cost_ab_4x50_stream_ttft_reasoning_default_length_tiers_chat_completions_benchmark_requests.jsonl</code>
Filtered records	<code>export/minimax_m27_all_experiments/routing_sort_cache_cost_ab_4x50_stream_ttft_reasoning_default_length_tiers_chat_completions_records.json</code>
Excluded–record audit	<code>export/minimax_m27_all_experiments/routing_sort_cache_cost_ab_4x50_stream_ttft_reasoning_default_length_tiers_chat_completions_records_excluded.json</code>

Artifact	Path
Benchmark runner source	scripts/rerun_routing_sort_cache_cost_ab.py
HTML report renderer source	scripts/render_glm52_deepseek_style_report.py
Test source	tests/test_rerun_routing_sort_cache_cost_ab.py
Dataset reference	Built-in <code>controlled_cache_probe</code> with prompt-length tier construction; request-level export in <code>benchmark_request</code>